

第八篇：主交付链：Grilling → Spec → Tickets → Implement → Code Review

原创 AI追梦少年 FutureCraft AI 2026年10月1日 08:30 湖南

MATT POCKOCK • SKILLS

把一个 Epic 直接交给 Agent，会怎样？

交付不是一条命令
它是一串退出条件

grilling • spec • tickets • implement • code review

FutureCraft AI • Matt Pocock Skills 连载 • 2026.09

6 PARTS + CONCLUSION

PART 01

链路

不是瀑布

PART 02

澄清

先解决定

PART 03

Spec

守住边界

PART 04

Tickets

切出验证

PART 05

Implement

一票一会话

PART 06

Review

再做审计

先给结论

主交付链的价值不是强迫所有工作按直线走完，而是让“业务决定、边界、依赖、实现、质量”各自在最便宜的阶段暴露。哪个环节发现假设不成立，就回到该回去的阶段。

有人问：既然 Agent 已经很会写代码，为什么不把“支持 Billing Correction”直接扔给 `/implement` ？

因为这句话里至少混着五种不同的不确定性：业务规则有没有想清、术语是否一致、边界是否明确、工作能否分片、代码是否正确。

让一条命令同时解决它们，往往会得到一个很大的 diff，和一个很晚才出现的惊喜。

Matt Pocock Skills 当前公开的主线是：

Grilling → Spec → Tickets → Implement → Code Review

这不是敏捷瀑布。

它是一组不同的退出条件。每一段只处理最适合它的问题；发现问题属于上一段时，可以回退，而不是让实现阶段偷偷做产品或架构决定。

01 PART

1. Grilling：处理“我们到底要什么”

入口可以是 `/grill-me`，也可以是在代码库里工作的 `/grill-with-docs`。它们解决的是设计树上的未决分支。

退出条件不是“Agent 已经写了一份计划”，而是 shared understanding 已形成，人明确确认。

LedgerLoop 在这里确认：原始 Posted Invoice 不重写；第一期只处理付款后 30 天内的价格差额；refund 和 correction 是两条流程；税务重算不在第一期。

如果这些答案尚未存在，后面的所有产物都只是在放大猜测。

02 PART

2. Spec：处理“边界是什么”

to-spec 把已确认的对话和仓库事实组合成合同。它声明问题、解决方案、用户故事、实现与测试决定、非目标。

它的退出条件不是“文档写完了”，而是可以让一个没参加访谈的人说清：交付完成后有什么可观察的变化，测试从哪里验证，哪些合理能力明确不做。

当代码探索推翻了某项业务前提，不要在 Spec 里偷偷改掉。回到 grilling，让人确认新的决定。

03 PART

3. Tickets：处理“怎样以最小风险推进”

to-tickets 把 Spec 切成能够独立验证的 tracer-bullet 行为，并画出真实 blocker。

一张 ticket 的退出条件是：它足够小，能放进一次相对独立的实施会话；又足够完整，完成后不需要等待另一个“横向任务”才能证明它工作。

LedgerLoop 的第一张实现票不是 “做后台页面”，而是 “创建一笔幂等、可审计、不会修改原 invoice 的 correction”。它是第一个能站住的竖向切片。

04 PART

4. Implement：处理 “怎样在护栏内把这一票做完”

当前官方 README 把 `implement` 定位为：实现一份 spec 或一组 tickets 所描述的工作，在预先同意的 seam 上驱动 `/tdd`，提交前运行 `/code-review`。

它不是功能设计器。

对实现会话来说，最重要的纪律反而很朴素：

- 只拿一张已经 ready 的 ticket。
- 在已经确认的 seam 上写或更新测试。
- 经常运行类型检查和局部测试，不要积累到最后。
- 若发现前提不成立，带证据回到调查或澄清。
- 变更完成后，交给 review，而不是自己宣布 “应该没问题”。

“一票一会话”不是硬性格式，而是防止上下文越滚越大。会话越长，Agent 越容易把旧假设当事实，顺手扩大范围。相对独立的 ticket 让意图、验证和 diff 都更容易保持同一焦点。

05 PART

5. Code Review：处理 “做对了吗，以及做得好吗”

review 不应被理解为最后的礼仪性扫一眼。

它至少要回到来源 Spec/Ticket，核对实现是否忠实完成承诺；也要按仓库的工程规范检查设计、命名、重复和测试质量。第十篇会专门拆开这两个判断轴。

在这一篇，你只需要记住它的退出条件：不是“没有人吐槽”，而是来源约束和工程质量都经过了独立检查，发现的问题有明确去向。

06 PART 不是瀑布：回退是设计的一部分

主链看起来有方向，但它绝不要求一路向右。

例如实现 correction 时，Agent 发现支付服务无法区分原始交易和重试交易。它不是应该自己发明一个幂等策略，而是：

记录证据。

判断它影响的是实现细节、Spec 边界还是最初业务决定。

回到相应阶段。

如果只是缺一个实现 seam，补充 Spec；如果票的依赖被发现不真实，调整 tickets；如果“允许重复 correction 吗”根本没被确认，回到 grilling。

回退看上去会慢。

但它把返工放在代码还少、影响还小的时候发生。

07 PART LedgerLoop 的第一张交付记录

经过前面四个阶段，第一张 ticket 的记录应很短，但信息密度很高：

```
# Delivery Record: T2 Create idempotent billing correction

## Input
- Ticket: T2
- Source: Billing Correction v1 Spec

## Confirmed seam
- createBillingCorrection 的公开行为

## Change summary
- 创建 correction，不修改 Posted Invoice
- 同 idempotency key 返回同一结果
- 记录领域审计事件
```

```
## Verification
- 行为测试：原 invoice 保持不变
- 行为测试：重复提交没有第二笔 correction
- 类型检查与本地测试通过

## Next
- 进入双轴 code review；不提前接入 UI 和报表
```

最后一句很重要：不提前接 UI 和报表。

第一切片的价值，是先证明账务不变量和幂等这条最危险的链路。把它证明成立之后，界面、权限和读取模型才有可靠的地基。

08 反例：用大 diff 伪装完成

PART

“一个 Epic、六十个文件、三天完成”听起来很有生产力。

它也可能只是把澄清、设计、实现和验收全部压缩到一个不可审查的 diff 里。需求没确认的部分会被藏进默认值，架构没想清的部分会被藏进 helper，测试缺失会被一句“后续补”带过。

主交付链并不保证没有 bug。

它保证的是每一类风险都有一个更早、更便宜、可回退的处理位置。

09 结论

PART

Grilling、Spec、Tickets、Implement、Code Review 不是一套仪式。

它们共同把 Agent 的强项和人的责任放在正确位置：Agent 可以调查、综合、切片、实现和验证；人负责不可外包的业务判断、边界确认和风险取舍。

下一阶段，我们会把镜头对准反馈回路：测试为什么要围绕 seam，而不是覆盖率；review 为什么必须把“符合规范”和“符合 Spec”拆开判断。

资料：项目 README、`to-spec`、`to-tickets`。本文的 LedgerLoop 名称与示例为演示性虚构案例。

喜欢作者

Matt Pocock Skills 拆解连载 · 目录

上一篇

下一篇

2026/10/8 17:43

第八篇：主交付链：Grilling → Spec → Tickets → Implement → Code Review

第七篇：to-tickets：工单不是待办清单，而是一张可并行的验证图

第九篇：tdd：测试的单位是 seam，不是覆盖率